

```

#!/usr/bin/env python3
"""
Apache Worker 枯渴 再現ツール - GUI版
標準ライブラリ (tkinter) のみ使用
"""

import tkinter as tk
from tkinter import ttk, scrolledtext, messagebox
import threading
import socket
import time
import signal
import sys
import urllib.request
from datetime import datetime
from collections import defaultdict

# =====
# カラーテーマ (ターミナル風ダークUI)
# =====
BG = "#0d1117"
BG_PANEL = "#161b22"
BG_INPUT = "#21262d"
BG_LOG = "#010409"
ACCENT = "#f0883e"      # オレンジ (警告色)
ACCENT2 = "#3fb950"     # グリーン (正常)
ACCENT3 = "#58a6ff"     # ブルー (情報)
RED = "#f85149"
YELLOW = "#d29922"
FG = "#e6edf3"
FG_DIM = "#8b949e"
FG_LABEL = "#7ee787"
BORDER = "#30363d"
FONT_MONO = ("Courier", 10)
FONT_UI = ("Helvetica", 10)
FONT_TITLE = ("Helvetica", 13, "bold")
FONT_SMALL = ("Helvetica", 9)

# =====
# コア接続ロジック (CLIスクリプトから移植)
# =====
class KeepAliveConnection(threading.Thread):

```

```

def __init__(self, host, port, vhost, conn_id, keepalive_sec=3600):
    super().__init__(daemon=True)
    self.host = host
    self.port = port
    self.vhost = vhost
    self.conn_id = conn_id
    self.keepalive_sec = keepalive_sec
    self.sock = None
    self.connected_at = None
    self.status = "init"

def run(self):
    try:
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.settimeout(10)
        self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
        try:
            self.sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIIDLE, 60)
            self.sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIIDVL, 10)
            self.sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPCNT, 6)
        except AttributeError:
            pass
        self.sock.connect((self.host, self.port))
        self.connected_at = time.time()
        self.status = "connected"
        request = (
            f"GET / HTTP/1.1\r\n"
            f"Host: {self.vhost}\r\n"
            f"Connection: keep-alive\r\n"
            f"User-Agent: WorkerExhaustionTest/1.0\r\n\r\n"
        )
        self.sock.sendall(request.encode())
        self.sock.settimeout(15)
        response = b""
        try:
            while b"\r\n\r\n" not in response:
                chunk = self.sock.recv(4096)
                if not chunk:
                    break
                response += chunk
        except socket.timeout:
            pass
        self.status = "holding"
        deadline = time.time() + self.keepalive_sec
        while not _stop_event.is_set() and time.time() < deadline:
            try:
                self.sock.settimeout(30)

```

```

        time.sleep(30)
    except Exception:
        break
except ConnectionRefusedError:
    self.status = "refused"
except socket.timeout:
    self.status = "timeout"
except Exception:
    self.status = "error"
finally:
    self.status = "closed"
    if self.sock:
        try:
            self.sock.close()
        except Exception:
            pass

def close(self):
    if self.sock:
        try:
            self.sock.close()
        except Exception:
            pass

_stop_event = threading.Event()
_active_conns = []
_active_conns_lock = threading.Lock()
_stats = defaultdict(int)
_stats_lock = threading.Lock()

def fetch_server_status(host, port):
    try:
        url = f"http://{host}:{port}/server-status?auto"
        req = urllib.request.Request(url, headers={"Host": "localhost"})
        with urllib.request.urlopen(req, timeout=3) as resp:
            body = resp.read().decode()
            result = {}
            for line in body.splitlines():
                if ": " in line:
                    k, v = line.split(": ", 1)
                    result[k.strip()] = v.strip()
            return result
    except Exception:
        return None

```

```

def count_established(host, port):
    try:
        parts = host.split(".")
        hex_ip = "{:02X}{:02X}{:02X}{:02X}".format(
            int(parts[3]), int(parts[2]), int(parts[1]), int(parts[0])
        )
        hex_port = f"{port:04X}"
        target = f"{hex_ip}:{hex_port}"
        count = 0
        with open("/proc/net/tcp", "r") as f:
            for line in f.readlines()[1:]:
                p = line.split()
                if len(p) >= 4 and p[2] == target and p[3] == "01":
                    count += 1
        return count
    except Exception:
        return -1

```

```

# =====
# GUI アプリケーション
# =====

```

```

class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Apache Worker枯渴 再現ツール")
        self.configure(bg=BG)
        self.resizable(True, True)
        self.geometry("1000x780")
        self.minsize(900, 680)

        self._running = False
        self._start_time = None
        self._spawn_thread = None
        self._monitor_thread = None
        self._history_busy = []
        self._history_est = []
        self._history_time = []

        self._build_ui()
        self.protocol("WM_DELETE_WINDOW", self._on_close)

```

```

# -----
# UI構築
# -----
def _build_ui(self):

```

```

# タイトルバー
title_bar = tk.Frame(self, bg=BG_PANEL, pady=8)
title_bar.pack(fill="x")
tk.Label(title_bar, text="⚡ Apache Worker枯渇 再現ツール",
         font=FONT_TITLE, bg=BG_PANEL, fg=ACCENT).pack(side="left", padx=
tk.Label(title_bar, text="WAF → FW → Linux(Apache) セッションゾンビ検証",
         font=FONT_SMALL, bg=BG_PANEL, fg=FG_DIM).pack(side="left", padx=

tk.Frame(self, bg=BORDER, height=1).pack(fill="x")

# メインレイアウト (左:設定 / 右:ログ+グラフ)
main = tk.Frame(self, bg=BG)
main.pack(fill="both", expand=True, padx=0, pady=0)

left = tk.Frame(main, bg=BG_PANEL, width=320)
left.pack(side="left", fill="y", padx=0, pady=0)
left.pack_propagate(False)

right = tk.Frame(main, bg=BG)
right.pack(side="left", fill="both", expand=True)

self._build_left(left)
self._build_right(right)

def _build_left(self, parent):
    pad = {"padx": 14, "pady": 4}

    def section(text):
        f = tk.Frame(parent, bg=BG_PANEL)
        f.pack(fill="x", **{"padx": 0, "pady": 0})
        tk.Frame(f, bg=BORDER, height=1).pack(fill="x")
        tk.Label(f, text=f" {text}", font=("Helvetica", 9, "bold"),
                 bg=BG_PANEL, fg=FG_LABEL, anchor="w").pack(fill="x", padx=10)
        return f

    def row(parent, label, widget_fn):
        f = tk.Frame(parent, bg=BG_PANEL)
        f.pack(fill="x", padx=14, pady=3)
        tk.Label(f, text=label, font=FONT_SMALL, bg=BG_PANEL,
                 fg=FG_DIM, width=14, anchor="w").pack(side="left")
        w = widget_fn(f)
        w.pack(side="left", fill="x", expand=True)
        return w

    def entry(parent, default="", width=18):
        e = tk.Entry(parent, bg=BG_INPUT, fg=FG, insertbackground=FG,
                     relief="flat", font=FONT_MONO, width=width,

```

```

        highlightthickness=1, highlightcolor=ACCENT3,
        highlightbackground=BORDER)
    e.insert(0, default)
    return e

# — 接続先 —
s1 = section("🌐 接続先")
self.v_host = row(s1, "ホストIP", lambda p: entry(p, "192.168.1.10"))
self.v_port = row(s1, "ポート", lambda p: entry(p, "80", 6))
self.v_vhost1 = row(s1, "VHost 1", lambda p: entry(p, "vhost1.example.co
self.v_vhost2 = row(s1, "VHost 2", lambda p: entry(p, "vhost2.example.co

f_vhost2 = tk.Frame(s1, bg=BG_PANEL)
f_vhost2.pack(fill="x", padx=14, pady=(0, 4))
self.use_vhost2 = tk.BooleanVar(value=False)
tk.Checkbutton(f_vhost2, text="VHost 2 も同時に使用（複合枯渴）",
               variable=self.use_vhost2,
               bg=BG_PANEL, fg=FG_DIM, selectcolor=BG_INPUT,
               activebackground=BG_PANEL, font=FONT_SMALL).pack(anchor="w

# — モード —
s2 = section("⚙️ モード")
self.v_mode = tk.StringVar(value="normal")
modes = [("normal - 1秒ごとに接続追加（実事象再現）", "normal"),
         ("fast - 0.1秒ごとに接続追加（短時間検証）", "fast"),
         ("monitor - 接続生成なし、監視のみ", "monitor")]
for text, val in modes:
    tk.Radiobutton(s2, text=text, variable=self.v_mode, value=val,
                   bg=BG_PANEL, fg=FG, selectcolor=BG_INPUT,
                   activebackground=BG_PANEL, font=FONT_SMALL,
                   command=self._on_mode_change).pack(anchor="w", padx=14

# — パラメータ —
s3 = section("🔧 パラメータ")

def slider_row(parent, label, from_, to, default, unit=""):
    f = tk.Frame(parent, bg=BG_PANEL)
    f.pack(fill="x", padx=14, pady=3)
    tk.Label(f, text=label, font=FONT_SMALL, bg=BG_PANEL,
             fg=FG_DIM, width=14, anchor="w").pack(side="left")
    var = tk.IntVar(value=default)
    val_lbl = tk.Label(f, text=f"{default}{unit}", font=FONT_MONO,
                       bg=BG_PANEL, fg=ACCENT, width=8, anchor="e")
    val_lbl.pack(side="right")
    sl = tk.Scale(f, from_=from_, to=to, orient="horizontal",
                  variable=var, showvalue=False,
                  bg=BG_PANEL, fg=FG, troughcolor=BG_INPUT,

```

```

        highlightthickness=0, activebackground=ACCENT,
        command=lambda v: val_lbl.config(text=f"{v}{unit}"))
sl.pack(side="left", fill="x", expand=True)
return var

self.v_interval    = slider_row(s3, "接続間隔",    1,    60,    1,    "秒")
self.v_keepalive   = slider_row(s3, "接続保持",    60, 3600, 3600, "秒")
self.v_max_conn    = slider_row(s3, "最大接続数",   10,   500,   300, "本")
self.v_duration    = slider_row(s3, "実行時間",    60, 7200, 7200, "秒")
self.v_mon_intv    = slider_row(s3, "監視間隔",     2,   60,   10, "秒")

# — ステータスメーター —
s4 = section("リアルタイム状態")
self._meters = {}
for key, label, color in [
    ("busy",    "BusyWorkers",  ACCENT),
    ("idle",    "IdleWorkers",  ACCENT2),
    ("est",     "ESTABLISHED",  ACCENT3),
    ("holding", "保持中接続数",  YELLOW),
    ("refused", "接続拒否数",    RED),
]:
    f = tk.Frame(s4, bg=BG_PANEL)
    f.pack(fill="x", padx=14, pady=2)
    tk.Label(f, text=label, font=FONT_SMALL, bg=BG_PANEL,
              fg=FG_DIM, width=14, anchor="w").pack(side="left")
    lbl = tk.Label(f, text="--", font=("Courier", 12, "bold"),
                   bg=BG_PANEL, fg=color, width=8, anchor="e")
    lbl.pack(side="right")
    self._meters[key] = lbl

# — ボタン —
tk.Frame(parent, bg=BORDER, height=1).pack(fill="x", pady=(12, 0))
btn_f = tk.Frame(parent, bg=BG_PANEL)
btn_f.pack(fill="x", padx=14, pady=10)

self.btn_start = tk.Button(btn_f, text="▶ 開始", font=("Helvetica", 11, "k",
    bg=ACCENT2, fg="#0d1117", relief="flat",
    activebackground="#2ea043", cursor="hand2",
    command=self._start, padx=12, pady=6)
self.btn_start.pack(side="left", fill="x", expand=True, padx=(0, 4))

self.btn_stop = tk.Button(btn_f, text="■ 停止", font=("Helvetica", 11, "k",
    bg=RED, fg="white", relief="flat",
    activebackground="#b91c1c", cursor="hand2",
    command=self._stop, padx=12, pady=6,
    state="disabled")
self.btn_stop.pack(side="left", fill="x", expand=True, padx=(4, 0))

```

```

def _build_right(self, parent):
    # グラフキャンバス
    graph_frame = tk.Frame(parent, bg=BG, pady=0)
    graph_frame.pack(fill="x", padx=8, pady=(8, 0))

    tk.Label(graph_frame, text="BusyWorkers / ESTABLISHED 推移グラフ",
             font=FONT_SMALL, bg=BG, fg=FG_DIM).pack(anchor="w", padx=4)

    self.canvas = tk.Canvas(graph_frame, bg=BG_LOG, height=180,
                           highlightthickness=1, highlightbackground=BORDER)
    self.canvas.pack(fill="x", padx=4, pady=4)

    # ログエリア
    log_frame = tk.Frame(parent, bg=BG)
    log_frame.pack(fill="both", expand=True, padx=8, pady=(0, 8))

    tk.Label(log_frame, text="ログ出力",
             font=FONT_SMALL, bg=BG, fg=FG_DIM).pack(anchor="w", padx=4)

    self.log_box = scrolledtext.ScrolledText(
        log_frame, bg=BG_LOG, fg=FG, font=FONT_MONO,
        insertbackground=FG, relief="flat",
        highlightthickness=1, highlightbackground=BORDER,
        state="disabled", wrap="none"
    )
    self.log_box.pack(fill="both", expand=True, padx=4, pady=4)

    # ログのカラータグ
    self.log_box.tag_config("warn", foreground=ACCENT)
    self.log_box.tag_config("ok", foreground=ACCENT2)
    self.log_box.tag_config("info", foreground=ACCENT3)
    self.log_box.tag_config("error", foreground=RED)
    self.log_box.tag_config("dim", foreground=FG_DIM)
    self.log_box.tag_config("header", foreground=YELLOW)

    # -----
    # ログ出力
    # -----
    def _log(self, msg, tag=""):
        def _insert():
            self.log_box.config(state="normal")
            ts = datetime.now().strftime("%H:%M:%S")
            self.log_box.insert("end", f"[{ts}] {msg}\n", tag)
            self.log_box.see("end")
            self.log_box.config(state="disabled")
        self.after(0, _insert)

```



```

# -----
# モード変更時の間隔スライダー自動設定
# -----
def _on_mode_change(self):
    mode = self.v_mode.get()
    if mode == "fast":
        self.v_interval.set(1)    # スライダーは整数なのでfastは0.1秒をコード内で処理

# -----
# 開始
# -----
def _start(self):
    host = self.v_host.get().strip()
    port = self.v_port.get().strip()
    vhost = self.v_vhost1.get().strip()

    if not host or not vhost:
        messagebox.showerror("入力エラー", "ホストIPとVHost1は必須です")
        return
    try:
        port_int = int(port)
    except ValueError:
        messagebox.showerror("入力エラー", "ポートは数値を入力してください")
        return

    global _stop_event, _active_conns, _stats
    _stop_event.clear()
    _active_conns.clear()
    _stats.clear()
    self._history_busy.clear()
    self._history_est.clear()
    self._history_time.clear()

    self._running = True
    self._start_time = time.time()
    self.btn_start.config(state="disabled")
    self.btn_stop.config(state="normal")

    mode = self.v_mode.get()
    interval = 0.1 if mode == "fast" else self.v_interval.get()

    self._log("=" * 56, "header")
    self._log(f"  検証開始", "header")
    self._log(f"  ホスト      : {host}:{port_int}", "info")
    self._log(f"  VHost      : {vhost}", "info")
    if self.use_vhost2.get():

```

```

        self._log(f"  VHost2      : {self.v_vhost2.get()}", "info")
self._log(f"  モード      : {mode}   間隔={interval}秒", "info")
self._log(f"  保持時間   : {self.v_keepalive.get()}秒   最大接続={self.v_max_c
self._log("=" * 56, "header")

# モニタースレッド
self._monitor_thread = threading.Thread(
    target=self._monitor_loop,
    args=(host, port_int),
    daemon=True
)
self._monitor_thread.start()

if mode != "monitor":
    # 接続生成スレッド (VHost1)
    self._spawn_thread = threading.Thread(
        target=self._spawn_loop,
        args=(host, port_int, vhost, interval),
        daemon=True
    )
    self._spawn_thread.start()

# VHost2も使う場合
if self.use_vhost2.get():
    vhost2 = self.v_vhost2.get().strip()
    t2 = threading.Thread(
        target=self._spawn_loop,
        args=(host, port_int, vhost2, interval),
        daemon=True
    )
    t2.start()

# -----
# 停止
# -----
def _stop(self):
    _stop_event.set()
    self._running = False
    self.btn_start.config(state="normal")
    self.btn_stop.config(state="disabled")
    with _active_conns_lock:
        for c in _active_conns:
            c.close()
    self._log("停止しました", "warn")
    self._log_summary()

def _log_summary(self):

```

```

self._log("=" * 56, "header")
self._log("  結果サマリ", "header")
with _stats_lock:
    self._log(f"  接続拒否(503等): {_stats['refused']}回", "error" if _stat
    self._log(f"  タイムアウト    : {_stats['timeout']}回", "warn")
    self._log(f"  エラー          : {_stats['error']}回", "warn")
if _stats["refused"] > 0:
    self._log("  ✅ MaxRequestWorkers到達 → 事象再現成功!", "ok")
else:
    self._log("  ⓘ BusyWorkersの推移グラフを確認してください", "info")
self._log("=" * 56, "header")

# -----
# 接続生成ループ
# -----
def _spawn_loop(self, host, port, vhost, interval):
    conn_id = 0
    duration = self.v_duration.get()
    max_conn = self.v_max_conn.get()
    keepalive = self.v_keepalive.get()
    start = time.time()

    while not _stop_event.is_set():
        if time.time() - start >= duration:
            self._log(f"実行時間({duration}秒)経過 - 接続生成停止", "warn")
            break

        with _active_conns_lock:
            alive = sum(1 for c in _active_conns if c.is_alive())

        if alive >= max_conn:
            time.sleep(interval)
            continue

        conn = KeepAliveConnection(host, port, vhost, conn_id, keepalive)
        conn.start()
        with _active_conns_lock:
            _active_conns.append(conn)

        if conn_id % 20 == 0 and conn_id > 0:
            self._log(f"接続生成: {conn_id}本目 (VHost={vhost})", "dim")

        conn_id += 1
        time.sleep(interval)

# -----
# 監視ループ

```

```

# -----
def _monitor_loop(self, host, port):
    interval = self.v_mon_intv.get()

    while not _stop_event.is_set():
        with _active_conns_lock:
            holding = sum(1 for c in _active_conns if getattr(c, "status", ""))
        with _stats_lock:
            refused = _stats.get("refused", 0)
            timeout_cnt = _stats.get("timeout", 0)

        est = count_established(host, port)
        est_str = str(est) if est >= 0 else "N/A"

        srv = fetch_server_status(host, port)
        busy_str = idle_str = "N/A"
        if srv:
            busy_str = srv.get("BusyWorkers", "N/A")
            idle_str = srv.get("IdleWorkers", "N/A")

    # メーターを更新
    def _upd(b=busy_str, i=idle_str, e=est_str, h=holding, r=refused):
        self._meters["busy"].config(text=b)
        self._meters["idle"].config(text=i)
        self._meters["est"].config(text=e)
        self._meters["holding"].config(text=str(h))
        self._meters["refused"].config(text=str(r))

        # BusyWorkersの色を枯渇度に応じて変える
        try:
            bv = int(b)
            color = RED if bv >= 240 else (ACCENT if bv >= 180 else ACCEN
            self._meters["busy"].config(fg=color)
        except Exception:
            pass
    self.after(0, _upd)

    # ログ出力
    elapsed = int(time.time() - self._start_time) if self._start_time else
    e_str = f"{elapsed//3600:02d}:{(elapsed%3600)//60:02d}:{elapsed%60:02

tag = "ok"
try:
    bv = int(busy_str)
    if bv >= 240:
        tag = "error"
    elif bv >= 180:

```

```

        tag = "warn"
    except Exception:
        pass

    msg = (f"{e_str}  Busy={busy_str:>4}  Idle={idle_str:>4}  "
          f"EST={est_str:>4}  Hold={holding:>4}  拒否={refused}")
    if tag == "error":
        msg += "  △  枯渴寸前!"
    self._log(msg, tag)

    # グラフデータ更新
    try:
        self._history_busy.append(int(busy_str))
        self._history_est.append(int(est_str))
        self._history_time.append(elapsed)
        self.after(0, self._draw_graph)
    except Exception:
        pass

    # 拒否が出たら強調ログ
    if refused > 0:
        self._log(f"✅ 接続拒否({refused}回) MaxRequestWorkers到達を確認!",

    time.sleep(interval)

# -----
# グラフ描画
# -----
def _draw_graph(self):
    c = self.canvas
    c.delete("all")
    w = c.winfo_width()
    h = c.winfo_height()
    if w < 10 or h < 10:
        return

    pad_l, pad_r, pad_t, pad_b = 50, 20, 16, 28
    gw = w - pad_l - pad_r
    gh = h - pad_t - pad_b

    # グリッド
    for i in range(5):
        y = pad_t + gh * i // 4
        c.create_line(pad_l, y, w - pad_r, y, fill=BORDER, dash=(2, 4))

    # y軸ラベル (最大値を動的に決定)
    max_busy = max(self._history_busy) if self._history_busy else 256

```

```

max_busy = max(max_busy, 50)
for i in range(5):
    val = int(max_busy * (4 - i) / 4)
    y = pad_t + gh * i // 4
    c.create_text(pad_l - 4, y, text=str(val), anchor="e",
                  fill=FG_DIM, font=("Courier", 8))

# x軸ラベル
if self._history_time:
    max_t = self._history_time[-1] or 1
    for i in range(5):
        t = int(max_t * i / 4)
        x = pad_l + gw * i // 4
        label = f"{t//60}m" if t >= 60 else f"{t}s"
        c.create_text(x, h - pad_b + 10, text=label,
                      fill=FG_DIM, font=("Courier", 8))

def draw_line(data, color, label, y_off=0):
    if len(data) < 2:
        return
    pts = []
    n = len(data)
    for i, v in enumerate(data):
        x = pad_l + gw * i // max(n - 1, 1)
        y = pad_t + gh - int(gh * min(v, max_busy) / max_busy)
        pts.append((x, y))
    for i in range(len(pts) - 1):
        c.create_line(pts[i][0], pts[i][1], pts[i+1][0], pts[i+1][1],
                      fill=color, width=2)

# 凡例
c.create_text(pad_l + 6, pad_t + 10 + y_off, text=f"— {label}",
              anchor="w", fill=color, font=("Courier", 8))

draw_line(self._history_busy, ACCENT, "BusyWorkers", 0)
draw_line(self._history_est, ACCENT3, "ESTABLISHED", 14)

# 枯渴ライン (MaxRequestWorkers=256)
limit_y = pad_t + gh - int(gh * min(256, max_busy) / max_busy)
if 0 <= limit_y <= h:
    c.create_line(pad_l, limit_y, w - pad_r, limit_y,
                  fill=RED, dash=(4, 4), width=1)
    c.create_text(w - pad_r - 2, limit_y - 6, text="MaxReqWorkers",
                  anchor="e", fill=RED, font=("Courier", 7))

# -----
# 終了
# -----

```

```
def _on_close(self):
    _stop_event.set()
    with _active_conns_lock:
        for c in _active_conns:
            c.close()
    self.destroy()
```

```
# =====
if __name__ == "__main__":
    app = App()
    app.mainloop()
```